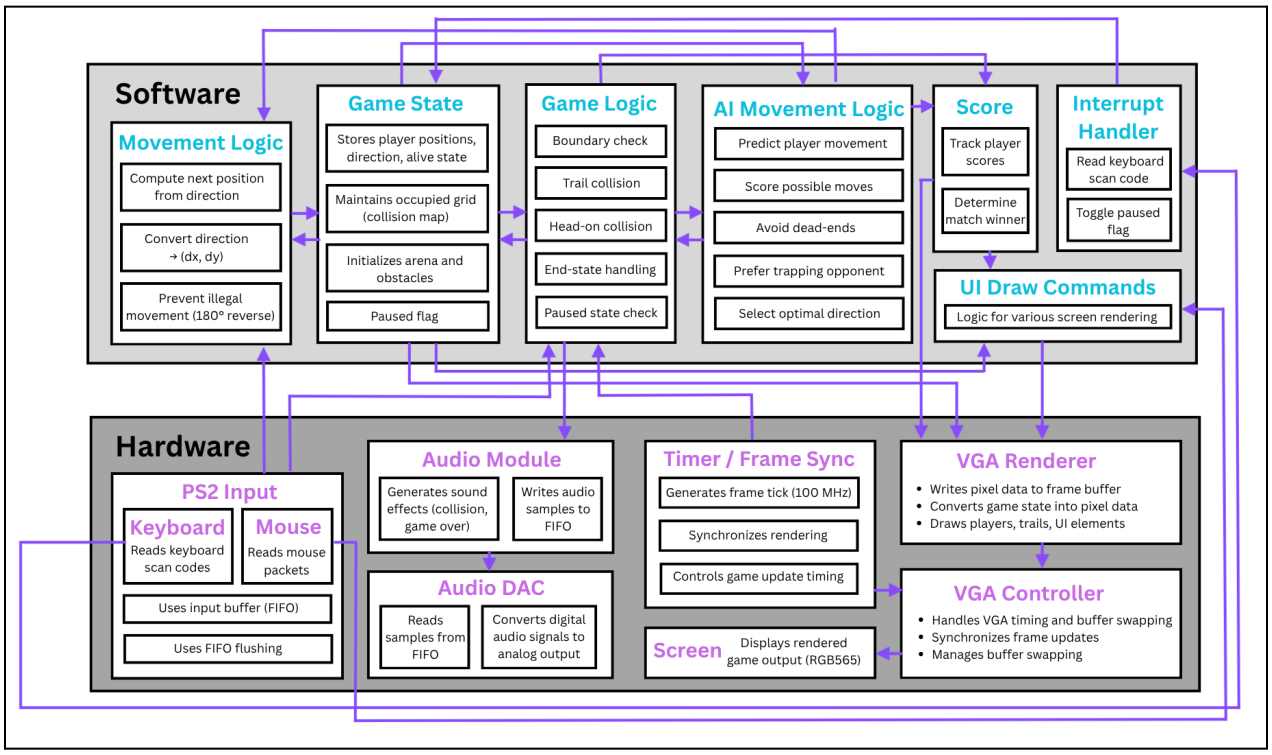
TRON

ECE243 Project Report *Hamda Armeen & Malak Khalifa*

1.0 Project Description

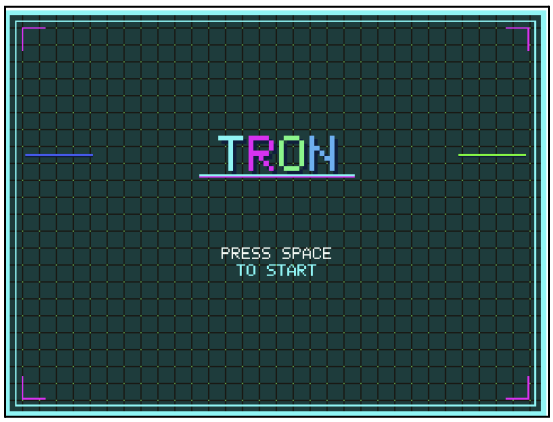
This project implements a real-time Tron-style light-cycle game on the DE1-SoC board. Two players, represented by colored squares (green and blue), leave persistent trails behind them as they move. The core gameplay loop updates at the VGA monitor's refresh rate. A player loses if their square collides with a trail, the screen border, an obstacle, or the opponent's square. The system features a multi-stage user interface, including a title screen with a start-up intro audio, a mode selection screen (multiplayer or single player where the second player is controlled by a predictive AI), a colour selection screen (single-player only) difficulty selection screen (easy, medium, hard), and a game-over screen with score display. Audio feedback is generated for collisions and game-ending events and pause/resume actions. The program supports two I/O devices: a PS/2 keyboard for player movement and a PS/2 mouse for menu navigation. Additionally, a PS/2 interrupt handler is implemented to capture the spacebar press for the pause functionality, ensuring immediate response even during active gameplay.

2.0 Block Diagram

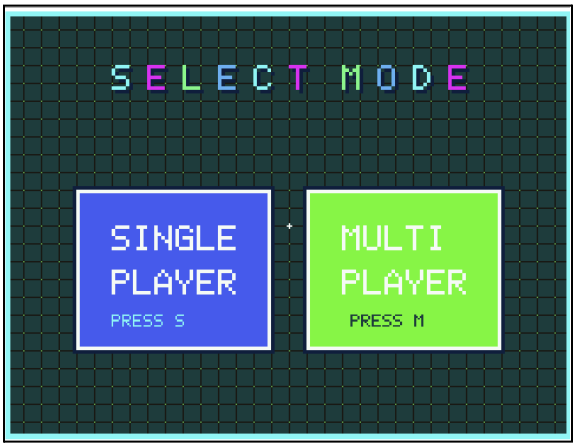


3.0 Project Operation

1. *Hardware Setup:*
    - Connect a PS/2 keyboard to the DE1-SoC's PS/2 port (primary).
    - Connect a PS/2 mouse to the DE1-SoC's PS/2 port (secondary).
    - Connect a VGA monitor to the board's VGA port.
    - Connect speakers or headphones to the audio line-out jack.
2. *Power On:*
    - Program the DE1-SoC with the provided code. The title screen will appear, and a startup sound will play.
3. *Start Game:*
    - Press the Spacebar on the keyboard OR left-click anywhere on the screen with the mouse to advance to the mode selection screen.



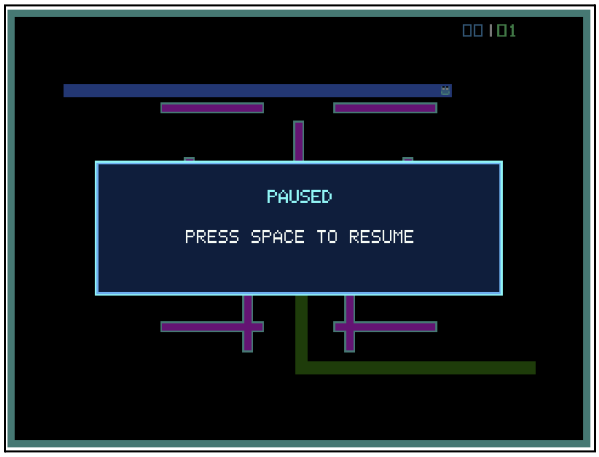
4. *Select Game Mode:*
    - For Single Player, press the *S* key.
    - For Two-Player, press the *M* key.
    - Alternately, drag the mouse cursor and left click within the bounds of the buttons on the screen.



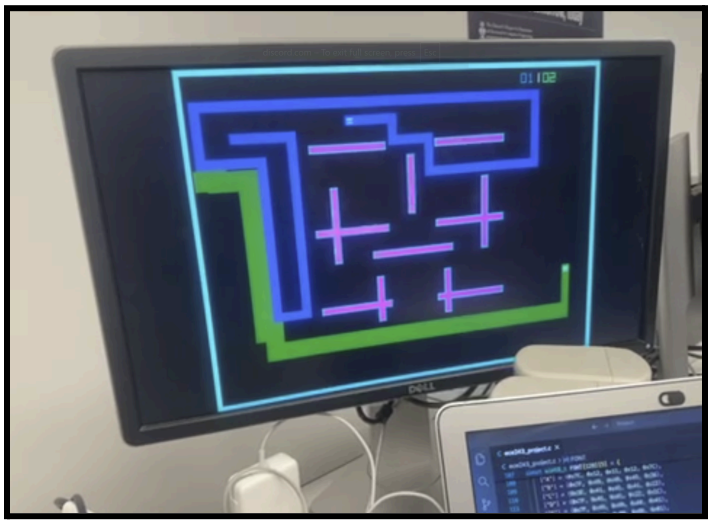
5. *Colour Selection (Single-Player Only):*
    - To select blue, press the *B* key.
    - To select blue, press the *G* key.
    - Alternately, drag the mouse cursor and left click within the bounds of the buttons on the screen.
    - To return to the previous mode selection screen, left click on the grey *back* button in the top left corner.

6. *Select Game Difficulty:*
    - For Easy, press key *1*.
    - For Medium, press key *2*.
    - For Hard, press key *3*.
    - Alternately, drag the mouse cursor and left click within the bounds of any of the buttons. (***Refer to Appendix A for all maze configurations***)

7. *Gameplay Controls:*
    - Player 1 (Blue): Uses *W* (up), *A* (left), *S* (down), *D* (right).
    - Player 2 (Green): Uses *I* (up), *J* (left), *K* (down), *L* (right).
    - Pause: Press the Spacebar at any time during gameplay to pause. A visual banner and audio cue will confirm the action.



    - Resume: Press Spacebar again to resume.
    - Quit: Press *Q* on the keyboard to return to the title screen at any time.
8. *Gameplay Rules:*
    - Control your dot, leaving a trail.
    - Avoid hitting walls, your own trail, the opponent's trail, and the opponent's dot.
    - The game continues until a player scores 3 points by causing the opponent to crash.



Player VS AI (green)

9. *End of Round:*
    - After a crash, a sound effect plays, the score updates, and the round resets.
10. *Game Over:*
    - When a player reaches 3 points, a "GAME OVER" screen appears, showing the final score and the winner.
    - Press the *Spacebar* to return to the title screen and start a new game.

AI wins (Green score)

4.0 Attribution Table

Task	*Task Description*	*Name*
Pixel Movement Logic	- Implemented grid-based movement using directional encoding (0–3 for up/right/down/left). - Developed compute_next_location() to calculate next player coordinates each frame. - Enforced movement constraints such as no 180° reversal using dirs_are_opposite(). - Integrated movement with frame-based timing using wait_for_video_frame() to ensure consistent speed.	Malak Khalifa

Collision Detection	- Designed the full collision system using a 2D occupancy grid (occupied[][]).
	- Implemented check_collision() to detect:
	  - Boundary (out-of-bounds) collisions
	  - Obstacle collisions
	  - Trail collisions
	  - Head-on player collisions
	- Created 5×5 footprint collision checking to match rendered trail thickness (not just a single pixel).
		Hamda Armeen

Game State & Round Manageme nt	- Designed core game state using:
	  - Player struct (position, direction, alive state)
	  - Global occupancy grid
	- Implemented setup_round() to:
	  - Reset board state
	  - Initialize player spawn positions
	  - Generate arena borders
	- Controlled transitions between:
	  - Title screen
	  - Mode selection
	  - GameplayGame over
		Malak Khalifa

Trail Memory & Dynamic Walls	- Implemented persistent trail system using occupied[][].
	- Designed avoid_self_collision() to:
	  - Convert previous player positions into permanent obstacles
	  - Avoid marking overlap with new head position (prevents instant self-collision)
	- Ensured trails visually match collision logic.
		Malak Khalifa

Game Logic (Endgame, Restart, Flow Control)	- Implemented win/loss conditions based on collisions.
	- Designed score tracking and win threshold (first to 3 points).
	- Handled:
	  - Round reset after collision
	  - Full game reset after win
	  - Restart using space bar input
	  - Quitting (return to home screen) with Q key
		Hamda Armeen

AI Movement Logic	- Developed predictive AI using ai_move():
	  - Predicts future player positions (1–3 steps ahead)
	  - Evaluates candidate moves (straight/left/right)
	- Implemented weighted scoring system using ai_advanced_movement():
	  - Prioritizes interception paths using Manhattan distance
	- Ensured AI respects:
	  - Collision constraints
	  - No illegal reverse movement
	- Integrated seamlessly with real-time game loop.
		Hamda Armeen

Difficulty Modes	- Designed multiple gameplay difficulty levels to adjust challenge and map complexity.	Malak Khalifa

	- Implemented three modes:
	  - Easy: Open arena with no internal obstacles, allowing free movement.
	  - Medium: Introduced static obstacle blocks placed strategically using fill_obstacle_rectangle() to restrict movement paths.
	  - Hard: Created a maze-like structure using wall segments, forcing tighter navigation and increasing collision probability.
	- Integrated difficulty selection through both keyboard input (1/2/3) and mouse-based UI buttons, with the selected mode dynamically configuring the arena during setup_round().	

Screen Displays	- Built full VGA rendering system using:
	  - draw_pixel() with direct memory-mapped I/O
	- Designed reusable rendering functions:
	  - draw_trail() (player + glow effect)
	  - clear_screen()
	  - fill_rectangle()
	- Implemented complete UI screens:
	  - Title screen
	  - Mode selection screen
	  - Game over screen
	- Created a consistent visual theme using custom colors and borders.
		Hamda Armeen

Font & Text Rendering Engine	- Implemented custom bitmap font system (5×7 grid).
	- Built:
	  - draw_char()
	  - draw_string()
	  - draw_string_scaled() for large UI text
	- Used for:
	  - Title (“TRON”)
	  - Instructions
	  - Game over messages
		Hamda Armeen

Score Display	- Designed score rendering using draw_digit():
	  - Custom pixel-based number rendering
	- Implemented draw_score():
	  - Displays both players’ scores in real time
	  - Color-coded (blue vs green)
	- Extracted digits manually (no libraries used).
		Hamda Armeen

VGA Set Up	- Configured VGA hardware using memory-mapped registers.
	- Managed pixel buffer access via:
	  - pixel_buffer_start
	- Implemented double buffering synchronization using:
	  - wait_for_video_frame()
	- Ensured flicker-free rendering and stable frame updates.
		Malak Khalifa

PS/2 Set Up	- Implemented low-level keyboard input using memory-mapped I/O.
	- Developed:
	  - keyboard_input()
	  - poll_ps2_keyboard()
	- Handled PS/2 protocol specifics:
	  - Break codes (0xF0)
	  - Make codes for key presses
	- Mapped keys to player directions (WASD, IJKL).
		Malak Khalifa

Audio Set Up	- Implemented audio output using WM8731 audio codec.
	- Developed:
	  - audio_push() for FIFO-based audio writing
	- Created sound effects:
	  - Start screen melody
	  - Collision sound
	  - Game over sound
	- Designed square wave generator using:
	  - play_square_tone() with 8 kHz sampling
	- Integrated audio feedback during collisions and game over
		Malak Khalifa

Mouse Set Up	- Implemented low-level PS/2 mouse input using memory-mapped I/O (port B).
	- Developed:
	  - mouse_init() to reset and enable mouse streaming
	  - mouse_poll() to read and decode 3-byte PS/2 packets
	  - Real-time tracking of cursor position with boundary clamping
	  - Detection of left-click events using button state transitions
	- Built a custom cursor system:
	  - draw_cursor() renders a cross
	  - erase_cursor() restores previous pixels to
		Hamda Armeen

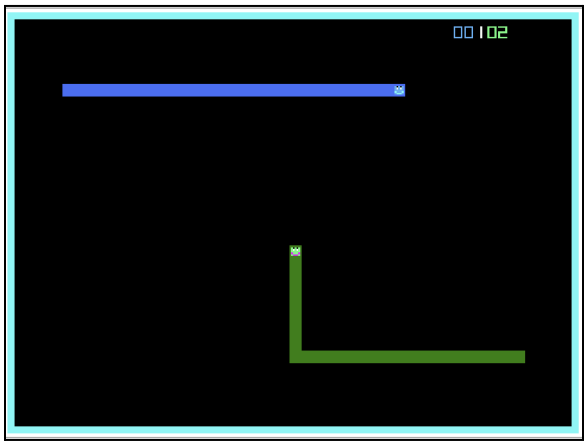
	prevent visual artifacts
	- Integrated mouse interaction with UI screens:
	  - Mode selection
	  - Colour selection
	  - Difficulty selection
	  - Return to previous screen with back button
	- Implemented by detecting cursor overlap with button regions and triggering selections on click	

PS/2 Interrupt	- Configured the PS/2 primary port to generate interrupts. - Wrote ps2_interrupt_handler() as an interrupt service routine (ISR) to capture the spacebar make code (0x29) asynchronously. - Used inline assembly to set the interrupt vector (mtvec), enable the PS/2 interrupt bit in mie, and enable global interrupts in mstatus.	Hamda Armeen & Malak Khalifa (Collaborative)

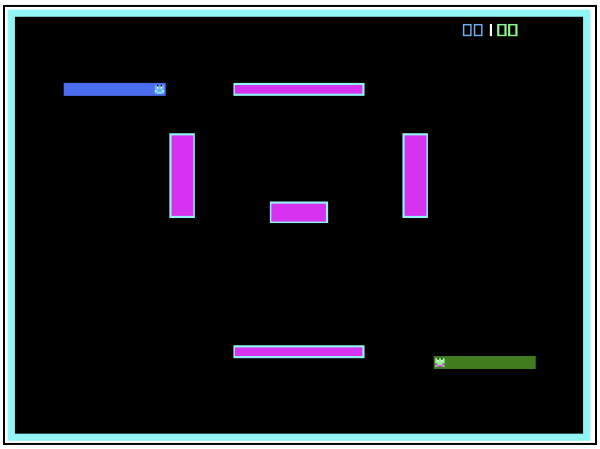
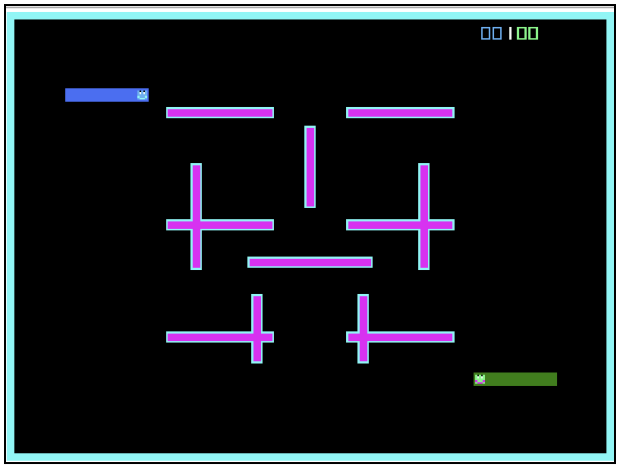
Total # Tasks by Hamda Armeen: 7.5 (50%)
Total # Tasks by Malak Khalifa: 7.5 (50%)
Both completed tasks with overall equal work load and difficulty.

Appendix A

Easy (Empty arena):



Medium (minor obstacles):



Hard (Multiple obstacles = maze)